Queue

Data Structure and Algorithms with C++

Queue Data Structure: Types, Implementation, Applications

A queue is a linear data structure that stores the elements sequentially. It uses the FIFO approach (First In First Out) for accessing elements. It is an abstract data type(There is a specific order to insert and retrieve elements).

Queues are typically used to manage threads in multithreading and implementing priority queuing systems.

A queue is an important data structure in programming. A queue follows the FIFO (First In First Out) method. A queue is open at both its ends. One end (rear end or the tail) is always used to insert data (enqueue) and the other end(front end or the head) is used to remove data (dequeue).

Some other applications of the Queue

Applications of the queue in real-life are:

- People on an escalator
- Cashier line in a store
- A car wash line
- One way exits

Types of Queues in Data Structure

There are four different types of queues in data structures:

- Simple Queue
- Circular Queue
- Priority Queue
- Double-Ended Queue (Deque)

Simple Queue

It is the most basic queue in which the insertion of an item is done at the end of the queue and deletion takes place at the front of the queue.

- Ordered collection of comparable data kinds.
- Queue structure is FIFO (First in, First Out).

Circular Queue

A circular queue is a special case of a simple queue in which the last member is linked to the first. As a result, a circle-like structure is formed.

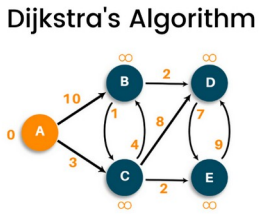
- The last node is connected to the first node.
- Also known as a **Ring Buffer** as the nodes are connected end to end.
- Insertion takes place at the front of the queue and deletion at the end of the queue.
- **Application of circular queue:** Insertion of days in a week

Priority Queue

In a priority queue, the nodes will have some predefined priority in the priority queue. The node with the least priority will be the first to be removed from the queue. Insertion takes place in the order of arrival of the nodes.

Some of the applications of priority queue:

- Dijkstra’s shortest path algorithm
- Prim’s algorithm
- Data compression techniques like Huffman code



Below diagram shows how an application use priority queue for the items consumed by the user.

Basic Operations in Queue Data Structure

Operation	Description
enqueue()	Process of adding or storing an element to the end of the queue
dequeue()	Process of removing or accessing an element from the front of the queue
peek()/front()	Used to get the element at the front of the queue without removing it
initialize()	Creates an empty queue
isfull()	Checks if the queue is full
isempty()	Check if the queue is empty

Enqueue Operation

Below are the steps to enqueue (insert) data into a queue

1. Check whether the queue is full or not.
2. If the queue is full – print the overflow error and exit the program.
3. If the queue is not full – increment the rear pointer to point to the next empty space.
4. Else add the element in the position pointed by Rear.
5. Return success.

Algorithm for Enqueue Operation

procedure enqueuer (data)

if queue is full

return overflow

endif

rear ← rear + 1

queue[rear] ← data

return true

end procedure

Dequeue Operation

Below are the steps to perform dequeue operation

- Check whether the queue is full or not.
- If the queue is empty – print the underflow error and exit the program.
- If the queue is not empty – access the data where the front is pointing.
- Else increment the front pointer to point to the next available data element.
- Return success.

Algorithm for Dequeue Operation

```
procedure dequeue
 if queue is empty
    return underflow
end if
data = queue[front]
front ← front + 1
return true
end procedure
```

Implementation of Queue

A queue can be implemented in two ways:

- Sequential allocation: It can be implemented using an **array**. A queue implemented using an array can organize only a limited number of elements.
- Linked list allocation: It can be implemented using a **linked list**. A queue implemented using a linked list can organize an unlimited number of elements.

Implementation using Arrays

In queue, insertion and deletion happen at the opposite ends.

To implement a queue using an array, create an array **arr** of size **n** and take two variables **front** and **rear** both of which will be initialized to 0 which means the queue is currently empty, not that in some implementation both rear and front can be initialized at -1, in this case you will need to understand real **when is the empty**. Element rear is the index up to which the elements are stored in the array and front is the index of the first element of the array. Now, some of the implementation of queue operations are as follows:

1. **Enqueue**: Addition of an element to the queue. Adding an element will be performed after checking whether the queue is full or not. If rear < n which indicates that the array is not full then store the element at arr[rear] and increment rear by 1 but if **rear == n** then it is said to be an Overflow condition as the array is full, but in this case if we have initialized the rear and the front to -1 then the overflow condition will be when **rear=n-1**.
2. **Dequeue**: Removal of an element from the queue. An element can only be deleted when there is at least an element to delete, i.e. rear > 0. Now, the element at arr[front] can be deleted but all the remaining elements have to be shifted to the left by one position in order for the dequeue operation to delete the second element from the left on another dequeue operation.
3. **Front**: Get the front element from the queue i.e. arr[front] if the queue is not empty.
4. **Display**: Print all elements of the queue. If the queue is non-empty, traverse and print all the elements from index front to rear.

1: Using normal Arrays

```
// C++, implement a queue using normal arrays
#include <iostream>
using namespace std;
int queue[100], n = 100, front = - 1, rear = - 1;
void enqueue() {
   int val;
   if (rear == n - 1)
   cout<<"Queue Overflow/Full"<<endl;
   else {
      if (front == - 1)
      front = 0;
      cout<<"Insert the element in queue : "<<endl;
      cin>>val;
      rear++;
      queue[rear] = val;
   }}
void dequeue() {
   if (front == - 1 || front > rear) {
      cout<<"Queue Underflow/Empty ";
      return ;
   }
      cout<<"Element deleted from queue is : "<<endl;
      // No need of shifting left , but this is not a better implementation
      front++;
}
```

```
void display() {
   if (front == - 1)
   cout<<"Queue is empty"<<endl;
   else {
      cout<<"Queue elements are : ";
      for (int i = front; i <= rear; i++)
      cout<<queue[i]<<" ";
                                    cout<<endl;
   }
}

void readFront(){
if(front==-1){
cout<<"Queue underflow"<<endl;
return ;

}
cout<<"Element at front
is:"<<queue[front]<<endl;

}
```

```
int main() {
   int ch;
   cout<<"1) Insert element to queue"<<endl;
   cout<<"2) Delete element from queue"<<endl;
   cout<<"3) Display all the elements of queue"<<endl;
   cout<<"4) Read element from the queue"<<endl;
   cout<<"5) Exit"<<endl;
   do {
      cout<<"Enter your choice : "<<endl;
      cin>>ch;
      switch (ch) {
                  case 1: enqueue();
                  break;
                  case 2: dequeue();
                  break;
                  case 3: display();
                  break;
                  case 4:readFront();
                 case 5: cout<<"Exited The Queue"<<endl;
                                    break;
                                    default:
cout<<"Invalid Operation"<<endl;
      }
   } while(ch>0 && ch<5);
   return 0;
}
```

Dequeue with shifting

```
    void dequeue() {
    if (front == - 1) {
       cout<<"Queue Underflow/Empty ";
       return ;
    }
       cout<<"Element deleted from queue is : "<< arr[front] <<endl;
    for( int i=front; i<=rear;i++){
arr[i]=arr[i+1];
}
rear--;
size--;
}
```

Example2: Pointer array and structure

```
#include <iostream>
using namespace std;
struct Queue{
int front, rear, n;
int *queue;
//constructor
Queue(int c){
front=rear=-1;
n=c;
queue=new int[n];
}
```

Example2: Pointer array and structure

```
#include <iostream>
using namespace std;
//int queue[100], n = 100, front = - 1, rear = - 1;
struct Queue{
int front, rear, n;
int *queue;
//constructor
Queue(int c){
front=rear=-1;
n=c;
queue=new int[n];
}
void enqueue() {
   int val;
   if (rear == n - 1)
   cout<<"Queue Overflow/Full"<<endl;
   else {
      if (front == - 1)
      front = 0;
      cout<<"Insert the element in queue : "<<endl;
      cin>>val;
      rear++;
      queue[rear] = val;
   }
}
void dequeue() {
   if (front == - 1 || front > rear) {
      cout<<"Queue Underflow/Empty ";
      return ;
   }
      cout<<"Element deleted from queue is : "<< queue[front] <<endl;
      // No need of shifting left , but this is not a better implementation
      front++;
```

```
void display() {
   if (front == - 1)
   cout<<"Queue is empty"<<endl;
   else {
      cout<<"Queue elements are : ";
      for (int i = front; i <= rear; i++)
      cout<<queue[i]<<" ";
                                          cout<<endl;
   }
}

void readFront(){
if(front==-1){
cout<<"Queue underflow"<<endl;
return ;

}
cout<<"Element at front
is:"<<queue[front]<<endl;

}};
```

```
int main() {
Queue  q(5);
   int ch;
   cout<<"1) Insert element to queue"<<endl;
   cout<<"2) Delete element from queue"<<endl;
   cout<<"3) Display all the elements of
queue"<<endl;
   cout<<"4) Read element from the queue"<<endl;
   cout<<"5) Exit"<<endl;
   do {
      cout<<"Enter your choice : "<<endl;
      cin>>ch;
      switch (ch) {
                        case 1: q.enqueue();
                        break;
                        case 2: q.dequeue();
                        break;
                        case 3: q.display();
                        break;
                        case 4:q.readFront();
                       case 5: cout<<"Exited The
Queue"<<endl;
                                          break;
                                          default:
cout<<"Invalid Operation"<<endl;
      }
   } while(ch>0 && ch<5);
   return 0;
}
```

Example 3:Generic Pointer array and structure

```
#include <iostream>
using namespace std;
template<typename T>
struct Queue{
int front, rear, n;
T *queue;
//constructor
Queue(int c){
front=rear=-1;
n=c;
queue=new T;
}
void enqueue() {
T val;
  if (rear == n - 1)
  cout<<"Queue Overflow/Full"<<endl;
  else {
  cout<<"Enter the element to insert";
  cin>>val;
                    if (front == - 1)
                    front = 0;
                    rear++;
                    queue[rear] = val;
  }
}

void dequeue() {
  if (front == - 1 || front > rear) {
    cout<<"Queue Underflow/Empty ";
    return ;
  }
    cout<<"Element deleted from queue is : "<< queue[front] <<endl;
    // No need of shifting left , but this is not a better implementation
    front++;
```

```
void display() {
  if (front == - 1)
  cout<<"Queue is empty"<<endl;
  else {
    cout<<"Queue elements are : ";
    for (int i = front; i <= rear; i++)
    cout<<queue[i]<<" ";
                                        cout<<endl;
  }
}

void readFront(){
if(front==-1){
cout<<"Queue underflow"<<endl;
return ;

}
cout<<"Element at front
is:"<<queue[front]<<endl;

}};
```

```
int main() {
Queue<double>  q(5);
  int ch;
  cout<<"1) Insert element to queue"<<endl;
  cout<<"2) Delete element from queue"<<endl;
  cout<<"3) Display all the elements of
queue"<<endl;
  cout<<"4) Read element from the queue"<<endl;
  cout<<"5) Exit"<<endl;
  do {
    cout<<"Enter your choice : "<<endl;
    cin>>ch;
    switch (ch) {
                    case 1: enqueue();
                    break;
                    case 2: dequeue();
                    break;
                    case 3: display();
                    break;
                    case 4:readFront();
                   case 5: cout<<"Exited The
Queue"<<endl;
                                        break;
                                        default:
cout<<"Invalid Operation"<<endl;
    }
  } while(ch>0 && ch<5);
  return 0;
}
```

Example 4: Using pointer Array with structure

```
// C++ program to implement a queue using a normal array, structure and pointer
#include <iostream>
using namespace std;
struct Queue {
                int front, rear, capacity;
// the array declared as a pointer!!
                int*  arr;
//constructor for the structure
                Queue(int c)
                {
                front = rear = 0;
                capacity = c;
                arr = new int;
                }

                // function to insert an element
                // at the rear of the queue
                void enqueue(int data)
                {
                // check queue is full or not
                if (capacity == rear) {
                cout<<endl<<"Queue is overflow"<<endl;
                return;
/ /INSERT ELEMENT
          arr[rear] = data;
         rear++;
                return;
                }
int frontF(){
return arr[front];

}
```

```
    // delete from the front of the queue
    void dequeue()
    {
                // if queue is empty
                if (front == rear) {
  cout<<endl<<"Queue is empty!"<<endl;
                return;
                }
                // shift all the elements from index 1(second element) till rear to the left by one
// notice the we do not reach at rear-1
                for (int i = 0; i < rear - 1; i++) {
                arr[i] = arr[i + 1];
                }
                // decrement rear
                rear--;
    }
    void display()
    {
                int i;
                if (front == rear) {
    cout<<endl<<"Queue is Empty!"<<endl;
                return;
                }
  // traverse front to rear and print elements
                for (i = front; i < rear; i++) {
                cout<<"   "<< arr[i];
                }
```

```
int main()
{
      Queue q(4);  // Create a queue of capacity 4
    q.display();    // print Empty Queue

    q.enqueue(10);  // inserting elements in the queue
    q.enqueue(300);
    q.enqueue(800);

    cout<<"Call Display when some 3 elements added"<<endl;
    // print Queue elements
    q.display();
    q.dequeue();
    q.dequeue();

 cout<<endl<< " Display after two node deletion"<<endl;

    // print Queue elements
    q.display();

    // print front of the queue
   int frontElement= q.frontF();

   cout<<endl<< "Reading the front element" <<endl<<" Front :"<<frontElement<<endl;

    return 0;
}
```

Example 5: Queue for any Data Type: Arrays

```
#include<iostream>
using namespace std;
//Use a template to make a queue of any data Type
template<typename T>
class Queue{
   T *arr;
   int nextIndex;
   int firstIndex;
   int size;
   int capacity;
   public:
            Queue(){
            capacity = 5;
            arr = new T[capacity];
            nextIndex = 0;
            firstIndex = -1;
            size = 0;
            }
            Queue(int cap){
            capacity = cap;
            arr = new T[capacity];
            nextIndex = 0;
            firstIndex = -1;
            size = 0;
            }
            /// insert element
            void enqueue(T ele){
            if(size == capacity){
            cout<<"Queue full"<<endl;
            return;
            }
            arr[nextIndex] = ele;
            nextIndex = (nextIndex + 1)%capacity;
            if(firstIndex == -1){
            firstIndex = 0;
            }
            size++;
            }
            int getSize(){
            return size;
            }
            bool isEmpty(){
            return size==0;
            }
T front(){
            if(isEmpty()){
            cout<<"Queue empty"<<endl;
            return 0;
            }
            return arr[firstIndex];
            }
            void dequeue(){
            if(isEmpty()){
            cout<<"Queue empty"<<endl;
            return;
            }
            firstIndex = (firstIndex +
1)%capacity;
            size--;


            }
};
```

```
int main(){

 Queue<int> q(5);
 q.enqueue(10);
 q.enqueue(20);
 q.enqueue(30);
 q.enqueue(40);
 q.enqueue(50);
 q.enqueue(60);
 q.enqueue(70);
 cout<<q.front()<<endl;
 q.dequeue();
 q.dequeue();
 q.dequeue();
 cout<<q.front()<<endl;
 cout<<q.getSize()<<endl;
 cout<<q.isEmpty()<<endl;
 q.enqueue(60);
 q.enqueue(70);

 q.dequeue();
 q.dequeue();
 cout<<q.front()<<endl;
 cout<<q.getSize()<<endl;
 return 0;
}
```

Example 6: Make a dynamic Queue using array

By using a no parameterised constructor, with the default capacity is 5; then whenever the size is equal to the capacity, it means the queue is full and we can insert the next element in the queue.

A quick solution is to create a new array by doubling the capacity of the array and copy all elements in the new array then delete the old array. See Example below:

Default Constructor

```
 template<typename T>
class Queue{
   T *arr;
   int rear;
   int front;
   int size;
   int capacity;

Queue(){
        capacity = 5;
        arr = new T[capacity];
        rear = 0;
        front = -1;
        size = 0;
      }
```

Get size(), isEmpty(), ifFull

```
int getSize(){
        return size;
    }
bool isEmpty(){
        return size==0;
    }
bool isFull(){
return size==capacity;
}
```

```
T front(){
        if(isEmpty()){
          cout<<"Queue empty"<<endl;
          return 0;
        }
        return arr[front];
    }
```

Dynamic Queue with array: Enqueue updated

By using a no parameterised constructor, the default capacity is 5; then whenever the size is equal to the capacity, it means the queue is full and we can insert the next element in the queue.

A quick solution is to create a new array by doubling the capacity of the array and copy all elements in the new array then delete the old array as per sample code on the left.

```
void enqueue(T ele){
        if(size == capacity){
          T *newArr = new T[2*capacity];
          int j = 0;
//Copy all elements in the array to the double array
          for(int i=front;i<capacity;i++){
              newArr[j] = arr[i];
              j++;
          }
          front=0;
          rear = capacity;
          capacity = 2*capacity;
          delete []arr;
          arr = newArr;
        }

        arr[rear] = ele;
        rear=rear+1;
        if(front == -1){
          front = 0;
        }
        size++;
      }
```

Dequeue with shifting

```
    void dequeue() {
    if (front == - 1) {
       cout<<"Queue Underflow/Empty ";
       return ;
    }
       cout<<"Element deleted from queue is : "<< arr[front] <<endl;
    for( int i=front; i<=rear;i++){
arr[i]=arr[i+1];
}
rear--;
size--;
}
```

Display

```
void display() {
  if (front == - 1)
  cout<<"Queue is empty"<<endl;
  else {
     cout<<"Queue elements are : ";
     for (int i = front; i <rear; i++)
     cout<<arr[i]<<" ";
                              cout<<endl;
  }
}
```

Implementation using Linked List

In the previous section, we introduced Queue and discussed array implementation. The following two main operations must be implemented efficiently.
In a Queue data structure, we maintain two pointers, front and rear. The front points the first item of the queue and the rear points to the last item.

1. enqueue() This operation adds a new node after it and moves to the next node.
2. dequeue() This operation removes the front node and moves front to the next node.

Implementation steps

- Create a class Node with data members integer data and Node* next
    - A parameterized constructor that takes an integer x value as a parameter and sets data equal to x and next as NULL
- Create a class Queue with data members Node front and rear
- Enqueue Operation with parameter x:
    - Initialize Node* temp with data = x
    - If the rear is set to NULL then set the front and rear to temp and return(Base Case)
    - Else set rear next to temp and then move rear to temp
- Dequeue Operation:
    - If the front is set to NULL return(Base Case)
    - Initialize Node temp with front and set front to its next
    - If the front is equal to NULL then set the rear to NULL
    - Delete temp from the memory

Implementation using the Linked list

```
#include<iostream>
using namespace std;
class Node{
   public:
      int data;
      Node* next;
      Node(int x){
        this->data = x;
        next = NULL;
      }
};
```

```
class Queue{
  Node* head;
    Node* tail;
    int size;

  public:
    Queue(){
        head = NULL;
        tail = NULL;
        size = 0;
    }
    int getSize(){
       return size;
    }
  int count(){
        …;
    }
    bool isEmpty(){
       return size==0;
    }
```

Enqueue, dequeue,display and front methods

```
void enqueue(int ele){
      Node* n = new Node(ele);
      if(head==NULL){
       head = n;
       tail = n;
      }else{
        tail->next = n;
        tail = n;
      }
      size++;
   }
```

```
void dequeue(){
      if(isEmpty()){
       return;
      }
      Node* temp = head;
      head = head->next;
      temp->next = NULL;
      delete temp;
      size--;
   }
```

```
      int front(){
         if(isEmpty()){
          return 0;
         }
         return head-
       >data;
         }
```

```
    void display(){
   Node *temp=head;
     while(temp!=NULL){
   cout<<temp->data<<" ";
   temp=temp->next;
      }
      }
};
```

```
Void display2(){
 while(!q.isEmpty()){
    cout<<q.front()<<endl;
    q.dequeue();
 }
}
```

Main method

```
int main(){
 Queue q;
 q.enqueue(10);
 q.enqueue(20);
 q.enqueue(30);
 q.enqueue(40);
 q.enqueue(50);
 q.enqueue(60);
 q.enqueue(70);
 q.display();
 cout<<q.front()<<endl;
 q.dequeue();
 q.dequeue();
 q.dequeue();
 cout<<q.front()<<endl;
 cout<<q.getSize()<<endl;
 cout<<q.isEmpty()<<endl;
 q.enqueue(60);
 q.enqueue(70);
 q.display();
 q.dequeue();
 q.dequeue();
 cout<<q.front()<<endl;
 cout<<q.getSize()<<endl;
//q.display2();//or the below
 while(!q.isEmpty()){
    cout<<q.front()<<endl;
    q.dequeue();
 }
 cout<<q.getSize()<<endl;
 cout<<q.isEmpty()<<endl;
   return 0;
}
```

Implementation using the Linked List and Generics

```
#include<iostream>
using namespace std;

template<typename T>
class Node{
   public:
      T data;
      Node<T>* next;
      Node(T data){
        this->data = data;
        next = NULL;
      }
};
```

```
template<typename T>
class Queue{
    Node<T>* head;
    Node<T>* tail;
    int size;

  public:
    Queue(){
        head = NULL;
        tail = NULL;
        size = 0;
    }
    int getSize(){
       return size;
    }
    bool isEmpty(){
       return size==0;
    }
```

Enqueue and dequeue

```
void enqueue(T ele){
    Node<T>* n = new Node<T>(ele);
    if(head==NULL){
     head = n;
     tail = n;
    }else{
      tail->next = n;
      tail = n;
    }
    size++;
  }
```

```
void dequeue(){
    if(isEmpty()){
     return;
    }
    Node<T>* temp = head;
    head = head->next;
    temp->next = NULL;
    delete temp;
    size--;
  }
```

Front and display

```
T front(){
    if(isEmpty()){
     return 0;
    }
    return head->data;
 }
```

```
void display(){
 Node<T> *temp=head;
   while(temp!=NULL){
cout<<temp->data<<" ";
temp=temp->next;
    }
    }
```

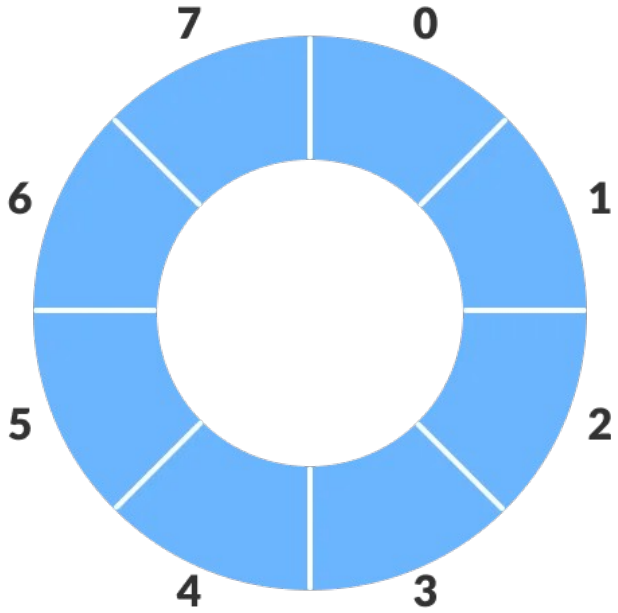
Main method

```
int main(){
 Queue<int> q;
 q.enqueue(10);
 q.enqueue(20);
 q.enqueue(30);
 q.enqueue(40);
 q.enqueue(50);
 q.enqueue(60);
 q.enqueue(70);
 q.display();
 cout<<q.front()<<endl;
 q.dequeue();
 q.dequeue();
 q.dequeue();
 cout<<q.front()<<endl;
 cout<<q.getSize()<<endl;
 cout<<q.isEmpty()<<endl;
 q.enqueue(60);
 q.enqueue(70);
 q.display();
 q.dequeue();
 q.dequeue();
 cout<<q.front()<<endl;
 cout<<q.getSize()<<endl;

 while(!q.isEmpty()){
    cout<<q.front()<<endl;
    q.dequeue();
 }
 cout<<q.getSize()<<endl;
 cout<<q.isEmpty()<<endl;
   return 0;
}
```

Circular Queue

Circular queue is the extended version of a regular queue where the last element is connected to the first element. Thus forming a circle-like structure.

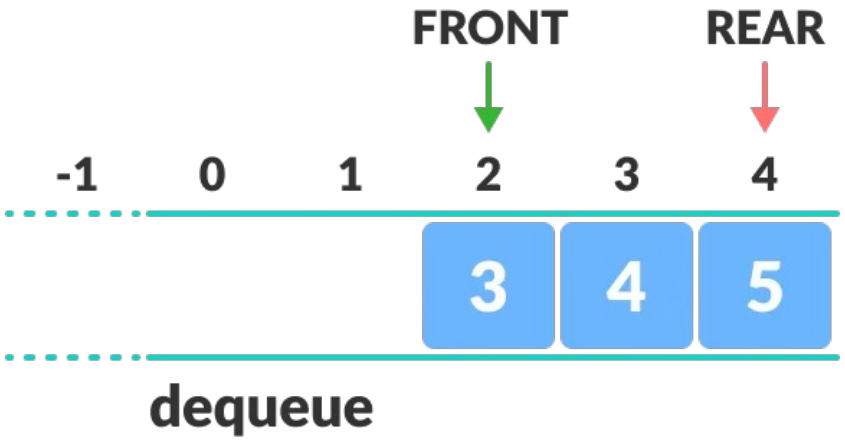


Limitation of the normal Queue vs Circular Queue

The circular queue solves the major limitation of the normal queue. In a normal queue, after a bit of insertion and deletion, **there will be non-usable empty space**.

Below is a limitation of a regular queue

While shifting index in dequeue operation, here, indexes 0 and 1 can only be used after resetting the queue (deletion of all elements). This reduces the actual size of the queue.



How circular Queue works

Circular Queue works by the process of circular increment i.e. when we try to increment the pointer and we reach the end of the queue, we start from the beginning of the queue.

Here, the circular increment is performed by modulo division with the queue size. That is,

```
if REAR + 1 == 5 (overflow!),
```

```
REAR = (REAR + 1)%5 = 0 (start of queue)
```

Circular queue operations

The circular queue work as follows:

- two pointers FRONT and REAR
- FRONT track the first element of the queue
- REAR track the last elements of the queue
- initially, set value of FRONT and REAR to -1

Enqueue and dequeue Operations

Enqueue operation

1. check if the queue is full
2. for the first element, set value of FRONT to 0
3. circularly increase the REAR index by 1 (i.e. if the rear reaches the end, next it would be at the start of the queue)
4. add the new element in the position pointed to by REAR

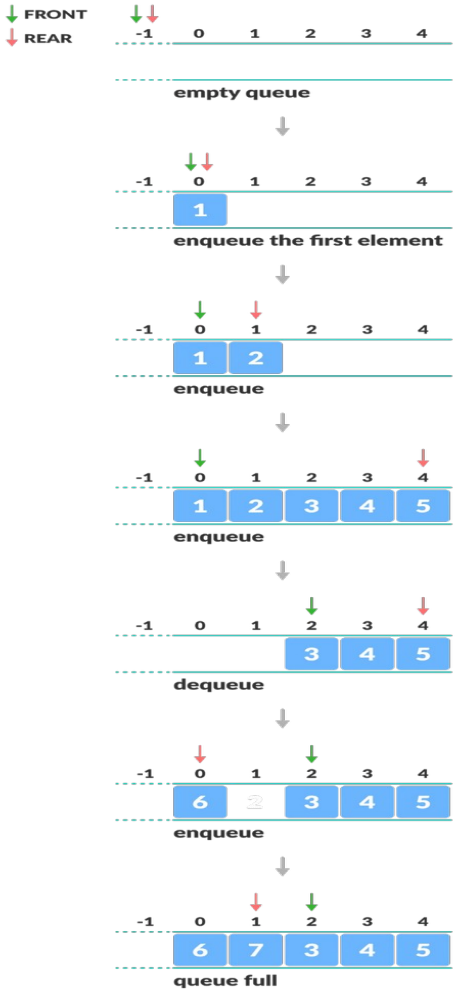
Dequeue operation

- check if the queue is empty
- return the value pointed by FRONT
- circularly increase the FRONT index by 1
- for the last element, reset the values of FRONT and REAR to -1

However, the check for full queue has a new additional case:

- Case 1: FRONT = 0 && REAR == SIZE - 1
- Case 2: FRONT = REAR + 1

The second case happens when REAR starts from 0 due to circular increment and when its value is just 1 less than FRONT, the queue is full.



Implementation of a circular Queue using Arrays

```
// Circular Queue implementation in C++

#include <iostream>
#define SIZE 5 /* Size of Circular Queue */

using namespace std;

class Queue {
  private:
 int items[SIZE], front, rear;

  public:
 Queue() {
  front = -1;
  rear = -1;
 }
 // Check if the queue is full
 bool isFull() {
  if (front == 0 && rear == SIZE - 1) {
   return true;
  }
  if (front == rear + 1) {
   return true;
  }
  return false;
 }
```

```
 // Check if the queue is empty
 bool isEmpty() {
  if (front == -1)
   return true;
  else
   return false;
 }
 // Adding an element
 void enQueue(int element) {
  if (isFull()) {
   cout << "Queue is full";
  } else {
   if (front == -1)
front = 0;
   rear = (rear + 1) % SIZE;
   items[rear] = element;
   cout << endl
     << "Inserted " << element <<
endl;
  }
```

```
 // Removing an element
 int deQueue() {
  int element;
  if (isEmpty()) {
   cout << "Queue is empty" <<
endl;
   return (-1);
  } else {
   element = items[front];
   if (front == rear) {
    front = -1;
    rear = -1;
   }
   // Q has only one element,
   // so we reset the queue after
deleting it.
   else {
    front = (front + 1) % SIZE;
   }
   return (element);
  }
```

Implementation(next…)

```
void display() {
  // Function to display status of Circular Queue
  int i;
  if (isEmpty()) {
    cout << endl
      << "Empty Queue" << endl;
  } else {
    cout << "Front -> " << front;
    cout << endl
      << "Items -> ";
    for (i = front; i != rear; i = (i + 1) % SIZE)
      cout << items[i];
    cout << items[i];
    cout << endl
      << "Rear -> " << rear;
  }
 }
};
```

```
int main() {
  Queue q;
  // Fails because front = -1
  q.deQueue();
  q.enQueue(1);
  q.enQueue(2);
  q.enQueue(3);
  q.enQueue(4);
  q.enQueue(5);
  // Fails to enqueue because front == 0 && rear == SIZE - 1
  q.enQueue(6);
  q.display();
  int elem = q.deQueue();
  if (elem != -1)
    cout << endl
      << "Deleted Element is " << elem;
  q.display();
  q.enQueue(7);
  q.display();
  // Fails to enqueue because front == rear + 1
  q.enQueue(8);
```

Circular Queue implementation using Linked List

The circular linked list is the example of implementation of a circular queue.

Revisit the circular LInked list program

Task:

1. Write the algorithm and implementation of Queue with the following details:

a) Circular using Linked list(Exercise)

b) Person Priority Queue using Linked list(Project). Student(code,name,age)